Web technology
Overview
Compiling from C/C++ to WebAssembly
Compiling an existing C module to WebAssembly
Compiling Text Rust to WebAssembly
Using the WebAssembly JavaScript API
Understanding WebAssembly text format
Converting WebAssembly text format
Loading and running WebAssembly code
Exporting WebAssembly functions
JavaScript bullets
Interpreted global entry constants
Reference
JavaScript interfaces

Compiling from Rust to WebAssembly

In this article we'll look at how to compile Rust to WebAssembly. We'll cover the basics of how to compile a Rust project into WebAssembly and use it in an existing web app.

Rust and WebAssembly use cases

There are two main use cases for Rust and WebAssembly:

- Write a standalone module to be used in a JavaScript application.
- Build a part of an application — using Rust or an existing JavaScript framework.

In the first, the Rust code is compiled into a single module, and in that's what we cover here. For the former case, Yew and Wasm-Bindgen are a good start.

In this article, we'll look at how to build a Text editing application, using Rust for handling JavaScript strings in Rust. This is a simple application, but it uses some non-trivial JavaScript code, so we won't cover Rust bindings. They may, on even notice that it's written in Rust.

Rust Environment Setup

Install Rust

Install Rust by using the the Quick Rust 7 page and following the instructions. This includes a tool called "rustup", which lets you manage multiple versions of Rust. By default, it installs the latest stable Rust release, which you can use for general Rust development. Rustup installs `rustc`, the Rust compiler, `cargo`, the Rust package manager, `rust-fmt`, Rust formatter, `rust-doc`, Rust documentation tool.

> **Note:** Pay attention to the post-install note about needing cargo's `bin` directory in your system `PATH`. This is added automatically, but you must restart your terminal for it to take effect.

wasm-pack

To build the package, we need an additional tool, `wasm-pack`. This takes the compiler code to WebAssembly and produces the right packaging for use in the browser. To download and install it, enter the following command into your terminal:

```
BASH
cargo install wasm-pack
```

Building our WebAssembly package

Now we can start to create a new package in Rust. Navigate to wherever you keep your projects, and type this:

```
BASH
cargo new --lib hello-wasm
```

This creates a new library in a subdirectory named `hello-wasm`, with everything you need to get started:

```
$ tree hello-wasm
hello-wasm
├── Cargo.toml
└── src
    └── lib.rs

2 directories, 2 files
```

`Cargo.toml`, is the file that configures our build. It works similarly to `package.json` for `npm` and `Cargo.lock` is a file that configures our build. It works similarly to `package.lock` for `npm`.

Cargo has also generated some Rust code for us in `src/lib.rs`:

```
RUST
pub fn add(left: usize, right: usize) -> usize {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
    }
}
```

Let's write some Rust

We won't use the generated `lib.rs` code shown above, replace it with the following:

```
RUST
pub mod utilities;

pub use utilities::*;

#[wasm_bindgen]
extern "C" {
    pub fn alert(s: &str);
}

#[wasm_bindgen]
pub fn greet(name: &str) {
    alert(&format!("Hello, {}!", name));
}
```

Our Rust code has three main parts, let's talk about each one in turn. We give a high level explanation here, and go into more detail in the next section. Rust's module system is beyond the scope of this article.

```
RUST
pub mod utilities;

pub use utilities::*;
```

Libraries are called "crates" in Rust.

Get it? Crate-crate-crates.

The first line creates a `mod` (module), which imports code from a library into your code. In this case, we're importing everything in the `mod utilities` module. We use these features in the next section.

Below we create a new in the next section, we should also make sure we import the `utilities` module.

```
RUST
#[wasm_bindgen]
extern "C" {
    pub fn alert(s: &str);
}
```

The next code block `#[wasm_bindgen]` attribute. In this case, it's not modifying an `extern` block, but a `fn`. This means that it's a function that we want to be publically accessible. It's the equivalent of `export`. These aren't the functions we need, but when the functions we're going to write are complete, we'll need to `export` them.

This is similar to `import`, but it's a function we are accepting rather than `export`-ing. Here, we import the `alert` function we passed in to the `extern` block above. It passes a call to the `Window` method, which is a function we import from the `wasm-bindgen` package.

The `extern "C"` means it takes two arguments in this case: a library and a variable to pass in. The format string is like `format` or `printf` in C-style libraries. `&str` is a string slice, which we introduced. The variable we're passing is `name`, the argument to the function, so if we set `greet("Chris")` we should see `"Hello, Chris!"`.

This is passed in `&str`, or when we call this function we will pass a string slice with "Hello, Sarah" or "Hello, Chris".

Now that our library is prepa, let's build it.

Compiling our code to WebAssembly

To compile our code correctly, we first configure it with `Cargo.toml`. Open this file, and change its contents to look like this:

```
TOML
[package]
name = "hello-wasm"
version = "0.1.0"
authors = ["Chris Mayer <chris@chris.com>"]
description = "A simple project with Wasm-Pack"
license = "MIT/Apache-2.0"
repository = "https://github.com/rustwasm/wasm-pack.git"
edition = "2018"

[lib]
crate-type = ["cdylib"]

[dependencies]
wasm-bindgen = "0.2.63"
```

We've added some descriptive and one key series info that `wasm-pack` uses for the `package.json`.

The big part to add is the `[lib]`section. The `[lib]` part tells Rust to build a `cdylib` version of our package, as wasm-pack gets what that means in the tutorial. For now, around the `cdylib` and `dylib` `crate-type`.

`[dependencies]` section. Here's where we tell Cargo what version of `wasm-bindgen` we want to depend on. In this case, we'll use `0.2.63`, version that not `0.2.0` or above.

Building the package

Now that we've completed the set up, let's build the package. We'll be using the generated code in a `pkg` subdirectory. Run `wasm-pack build` to build our package, and `wasm-pack build --target web` to specify what kind of WebAssembly and JavaScript is generated.

Finally, run the following command inside your `hello-wasm` directory:

```
BASH
wasm-pack build --target web
```

This does several things. To learn about them in detail, check out this learn page on Mozilla Hacks. In a nutshell, `wasm-pack build`:

1. Compiles your Rust code to WebAssembly.
2. Runs `wasm-bindgen` over that WebAssembly, generating a JavaScript file that wraps up the WebAssembly binary, and cleans up other outputs from the compiler.
3. Creates a `pkg` directory and moves that JavaScript file and your WebAssembly code into it.
4. Reads in your `Cargo.toml` and produces an equivalent `package.json`.
5. Copies your `README.md` if you have one into the package.

The end result? You have a package ready for npm distribution.

Using the package on the web

Now that we've got a complete Wasm module, let's run it in the browser. Let's start by creating a file named `index.html` in the root of the project, so we end up with the following project structure:

```
$ tree pkg hack
├── pkg
│   ├── README.md
│   ├── hello_wasm.d.ts
│   ├── hello_wasm.js
│   ├── hello_wasm.wasm
│   ├── hello_wasm_bg.d.ts
│   ├── hello_wasm_bg.wasm.d.ts
│   ├── hello_wasm_bg.wasm
│   ├── package.json
│   ├── pkg
│   ├── Cargo.lock
│   ├── Cargo.toml
│   ├── src
│   │   ├── lib.rs
│   │   ├── utilities
│   │   ├── utilities.rs
│   │   ├── utilities.test
│   │   ├── utilities.test.rs
│   │   ├── utilities.test.rs.bak
│   │   ├── utilities.test.rs.bak.1
│   │   ├── utilities.test.rs.bak.2
│   │   ├── utilities.test.rs.bak.3
│   │   ├── utilities.test.rs.bak.4
│   │   ├── utilities.test.rs.bak.5
│   │   ├── utilities.test.rs.bak.6
│   │   ├── utilities.test.rs.bak.7
│   │   ├── utilities.test.rs.bak.8
│   │   ├── utilities.test.rs.bak.9
│   │   ├── utilities.test.rs.bak.10
│   │   ├──
```